SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science & Engineering

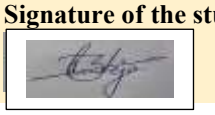
ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Course Code	**CSA4315**	**Course Title**	**INTERNET PROGRAMMING**
CO Assessed	**CO1**	**Assessment**	**Assessment Tool 1: Application Based Questions**
Weightage	**40% of CIA**	**Total Marks**	**100**
CO1 Statement	*Explain the fundamentals of web architecture, client-server communication, Internet protocols, HTTP request and response messages, and configure LAMP/WAMP environments for web application development and deployment.* **(BL3)**		
Student Name	**B. BHANUTEJA REDDY**	**Reg. No.**	**192325016**
Section/Batch	**Slot D / 4TH YEAR**	**Date**	**31-10-2026**

Code of Conduct:

I **B. BHANUTEJA REDDY**, certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Progressive Web Applications (PWA)

Architecture, Features, Working Principles, Service Workers, Manifest, Caching and Push Notifications

1. Introduction

A Progressive Web Application (PWA) is a web application built with standard web technologies (HTML, CSS and JavaScript) that uses modern browser APIs to deliver an app-like experience. A PWA is delivered over HTTPS, can be installed on the home screen, works offline or on poor networks, and can re-engage users with push notifications, while remaining a single code base that runs on every device with a modern browser.

The word "progressive" means the application works for every user as a basic web page and is progressively enhanced with richer capabilities (offline use, installation, notifications) wherever the browser supports them.

2. Features of a PWA

Feature	Description
Responsive	Layout adapts to mobile, tablet and desktop screens using fluid grids, flexible images and media queries.
Reliable (offline-first)	Loads instantly and works without a network because a service worker serves content from the cache.
Fast	App shell and static assets are cached, so repeat visits avoid network delays; smooth animations and quick interaction.
Installable	Can be added to the home screen / desktop via the Web App Manifest and runs in its own window without browser UI.
Engaging	Push notifications and background sync bring users back and keep data up to date.
Secure	Served only over HTTPS, which prevents tampering and is mandatory for service workers.
Discoverable	Identified as an "application" by the manifest and indexed by search engines, unlike store-only apps.
Linkable	Shared and opened through a simple URL, no store download or update approval needed.
Fresh / Cross-platform	Always updated in the background by the service worker; one code base for Android, iOS, Windows, macOS and Linux.

3. Architecture of a PWA

A PWA has a client-side architecture centred on three building blocks – the App Shell, the Service Worker and the Web App Manifest – supported by browser storage and notification APIs, and a conventional HTTPS back end.

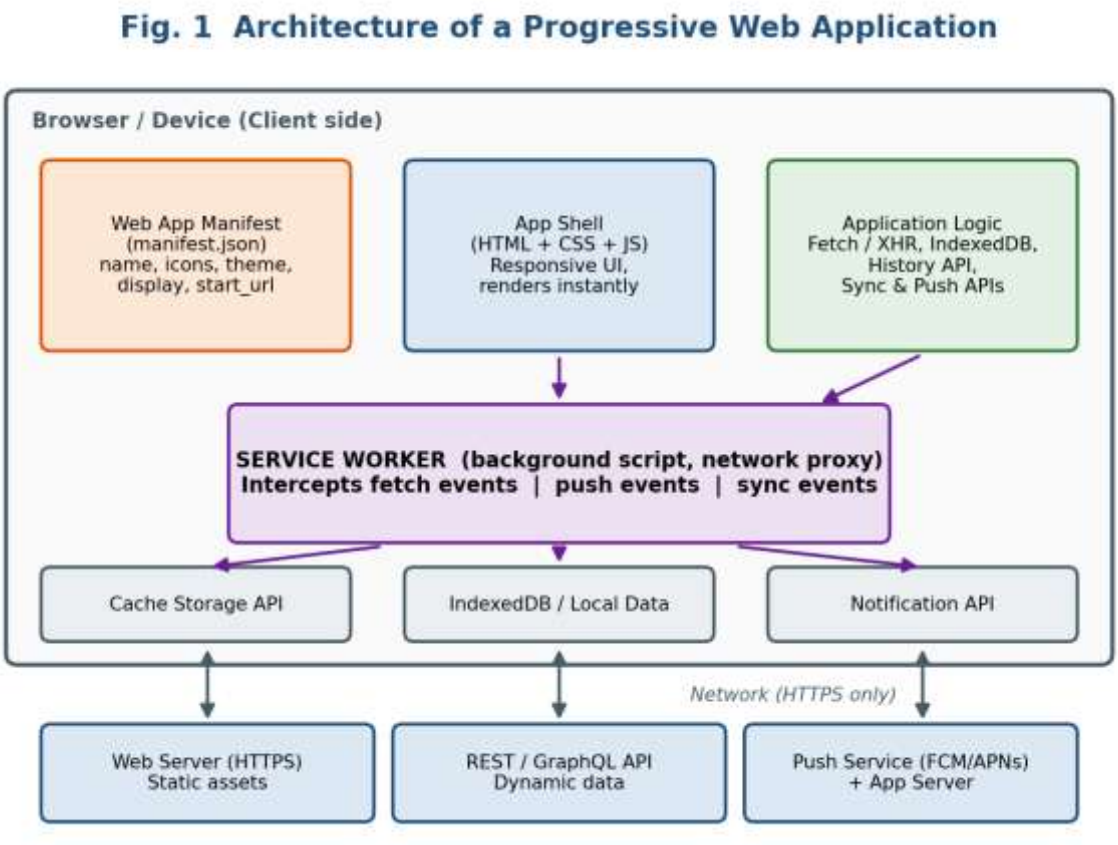


Fig. 1 Architecture of a Progressive Web Application

3.1 Components

- **Web App Manifest:** a JSON file that tells the browser how the app should look and behave when installed (name, icons, colours, start URL, display mode).
- **App Shell:** the minimal HTML, CSS and JavaScript needed to draw the user interface (header, navigation, layout). It is cached once and loads instantly; dynamic content is then filled in from the network or local storage.
- **Service Worker:** a JavaScript file that runs in a separate background thread and acts as a programmable network proxy between the page, the browser cache and the network. It handles fetch, push and sync events.
- **Storage layer:** Cache Storage API for request/response pairs and IndexedDB for structured application data.
- **Notification and Push APIs:** used to receive messages from the server through a browser push service and display them to the user.
- **HTTPS server and APIs:** deliver static files and dynamic data (REST/GraphQL) and send push messages.

3.2 Working Principle

The working of a PWA can be summarised in the following steps:

1. The user visits the URL for the first time; the browser loads the page, the manifest and the scripts from the server over HTTPS.
2. The page registers a service worker. The browser downloads it and fires the install event, in which the app shell and key assets are pre-cached.
3. After activation, the service worker controls the page. Every later request (fetch event) passes through the service worker, which decides whether to answer from the cache, the network or both, according to a caching strategy.

4. If the network is slow or absent, the cached shell and stored data are shown immediately, so the app still works offline.
5. Using the manifest, the browser offers an “Install / Add to Home screen” prompt. The installed app opens in a standalone window like a native app.
6. With the user’s permission, the app subscribes to push notifications; the server can then send messages that the service worker shows even when the app is closed.
7. Each time the service worker file changes, the browser installs the new version in the background and activates it, keeping the app fresh.

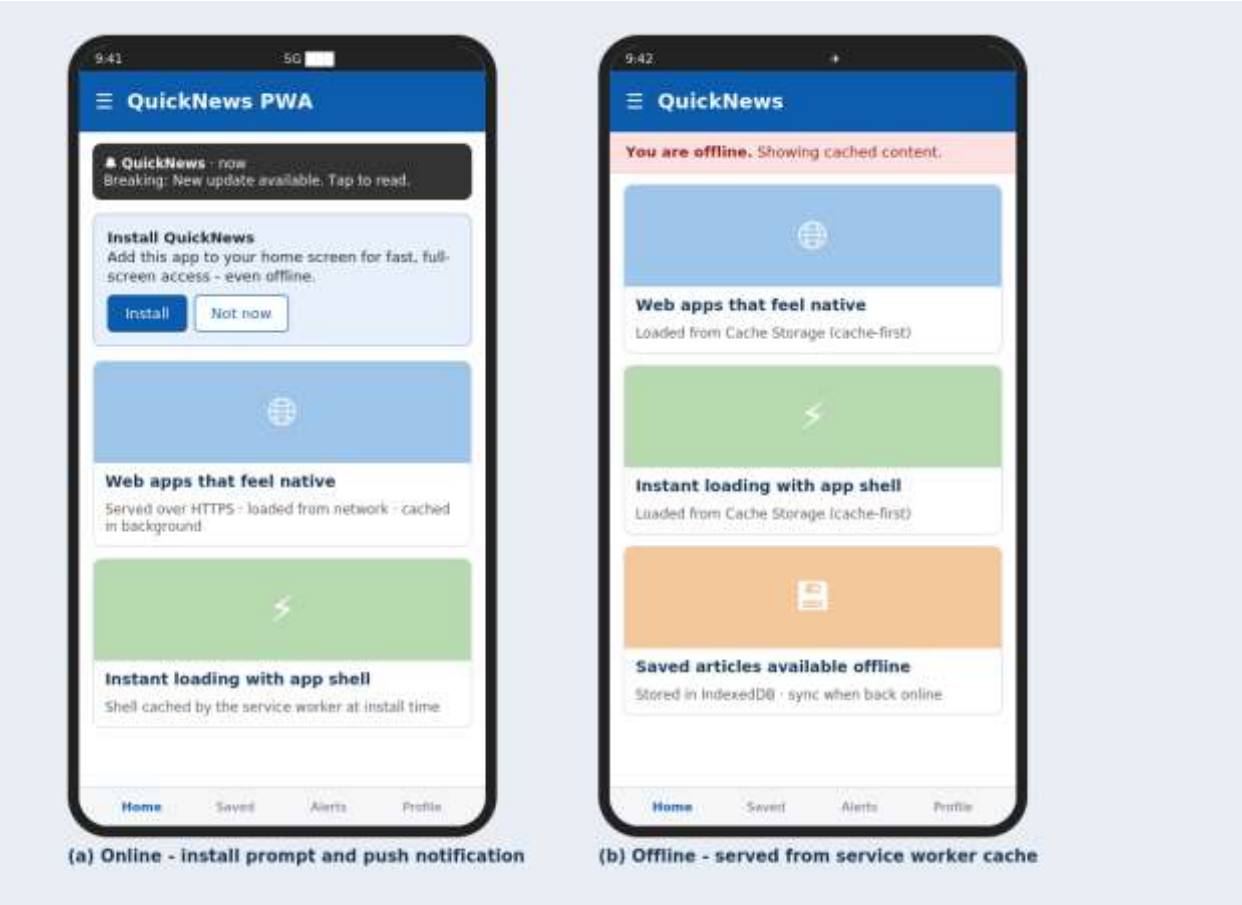


Fig. 2 Demo PWA web page – (a) online with install prompt and push notification, (b) offline, served from the service worker cache

4. Role of Web App Manifest

The Web App Manifest (manifest.webmanifest) is a JSON file linked from every page with <link rel="manifest" href="/manifest.webmanifest">. It makes the web app installable and controls its appearance on the device. Browsers (e.g. Chrome) use it, together with HTTPS and a service worker, to decide whether to show the install prompt.

```
{
 "name": "QuickNews",
 "short_name": "QuickNews",
 "start_url": "/index.html?source=pwa",
 "scope": "/",
 "display": "standalone",
 "orientation": "portrait",
 "theme_color": "#0B5CAD",
 "background_color": "#FFFFFF",
 "icons": [
   { "src": "/icons/icon-192.png", "sizes": "192x192", "type": "image/png" },
   { "src": "/icons/icon-512.png", "sizes": "512x512", "type": "image/png",
```

```
    "purpose": "any maskable" }
  ]
}
```

Member	Purpose
name / short_name	Full name for install dialogs and short name under the home-screen icon.
start_url / scope	Page opened when the app is launched, and the URL range that belongs to the app.
display	standalone, fullscreen, minimal-ui or browser – removes or keeps the browser address bar.
theme_color / background_color	Colour of the toolbar/status bar and of the splash screen while the app loads.
icons	Icons of different sizes for home screen, task switcher and splash screen.

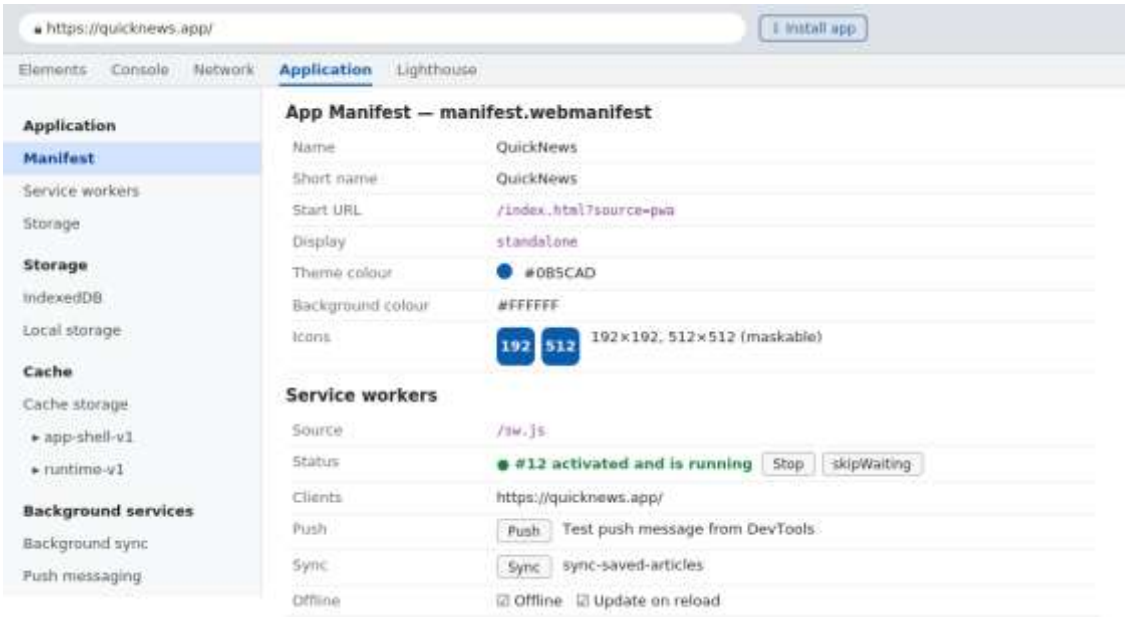


Fig. 3 Browser Application panel showing the manifest and an activated service worker

5. Role of Service Workers

A service worker is an event-driven JavaScript worker that runs in the background, separate from the main page. It has no direct access to the DOM, runs only on HTTPS (or localhost), and is the core technology behind offline support, caching, push notifications and background sync.

5.1 Registration

```
// main.js
if ('serviceWorker' in navigator) {
  window.addEventListener('load', () => {
    navigator.serviceWorker.register('/sw.js')
      .then(reg => console.log('SW registered, scope:', reg.scope))
      .catch(err => console.error('SW registration failed', err));
  });
}
```

5.2 Lifecycle

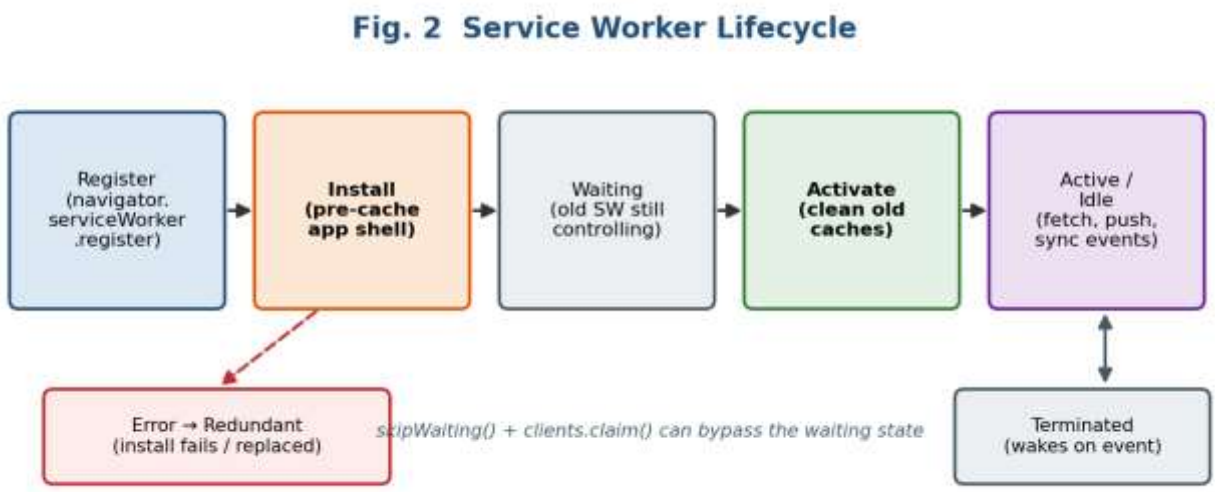


Fig. 4 Service worker lifecycle

- **Install:** fired once per new version; used to pre-cache the app shell. If it fails, the worker is discarded.
- **Waiting:** a new worker waits until pages controlled by the old version are closed (unless skipWaiting() is called).
- **Activate:** used to delete outdated caches and take control of clients (clients.claim()).
- **Idle / Terminated:** the browser stops the worker when idle and wakes it for a fetch, push or sync event.

5.3 Functions of the service worker

- Intercepts and handles network requests (fetch event) – enables offline use and custom caching.
- Pre-caches the app shell during install and manages cache versions during activate.
- Receives push messages and displays notifications (push, notificationclick events).
- Performs background sync so that actions taken offline are sent when connectivity returns.

```
// sw.js  - skeleton service worker
const SHELL = 'app-shell-v1';
const FILES = ['/', '/index.html', '/styles.css', '/app.js', '/offline.html',
               '/icons/icon-192.png'];

self.addEventListener('install', e => {
  e.waitUntil(caches.open(SHELL).then(c => c.addAll(FILES)));
  self.skipWaiting();
});

self.addEventListener('activate', e => {
  e.waitUntil(caches.keys().then(keys => Promise.all(
    keys.filter(k => k !== SHELL).map(k => caches.delete(k)))));
  self.clients.claim();
});

self.addEventListener('fetch', e => {
  e.respondWith(
    caches.match(e.request).then(hit => hit || fetch(e.request))
      .catch(() => caches.match('/offline.html'))
  );
});
```

6. Caching Strategies

Caching strategies define how the service worker responds to a request. The right choice depends on how often the resource changes and how important freshness is compared with speed.

Fig. 3 Service Worker Caching Strategies

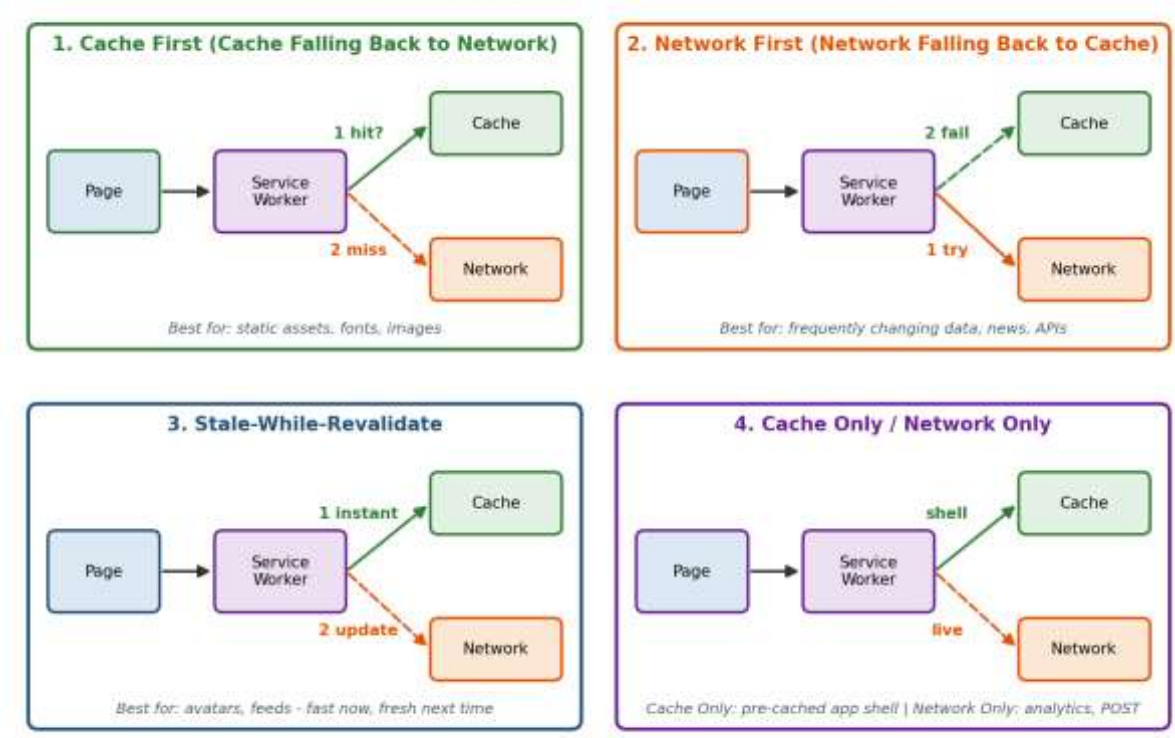


Fig. 5 Service worker caching strategies

Strategy	How it works	Suitable for
Cache First	Looks in the cache first; goes to the network only on a miss and then stores the response.	Static assets: CSS, JS, fonts, logos, images.
Network First	Tries the network; if it fails or times out, falls back to the cached copy.	Frequently changing content: news, dashboards, API data.
Stale-While-Revalidate	Returns the cached copy immediately and updates the cache from the network in the background.	Avatars, feeds, non-critical data needing speed and eventual freshness.
Cache Only	Always answers from the cache (pre-cached during install).	Versioned app shell files.
Network Only	Always goes to the network, no caching.	Analytics pings, POST requests, payments.

Example – Stale-While-Revalidate:

```
self.addEventListener('fetch', event => {
  event.respondWith(
    caches.open('runtime-v1').then(async cache => {
      const cached = await cache.match(event.request);
      const network = fetch(event.request).then(res => {
        cache.put(event.request, res.clone());   // refresh cache
        return res;
      });
      return cached || network;                  // fast: cached first
```

```
  })
 );
});
```

7. Push Notifications

Push notifications let a PWA re-engage users even when the app is not open. Two APIs cooperate: the Push API (delivers a message from the server to the service worker through a browser push service such as FCM, APNs or Mozilla Push) and the Notification API (shows the message on the device).

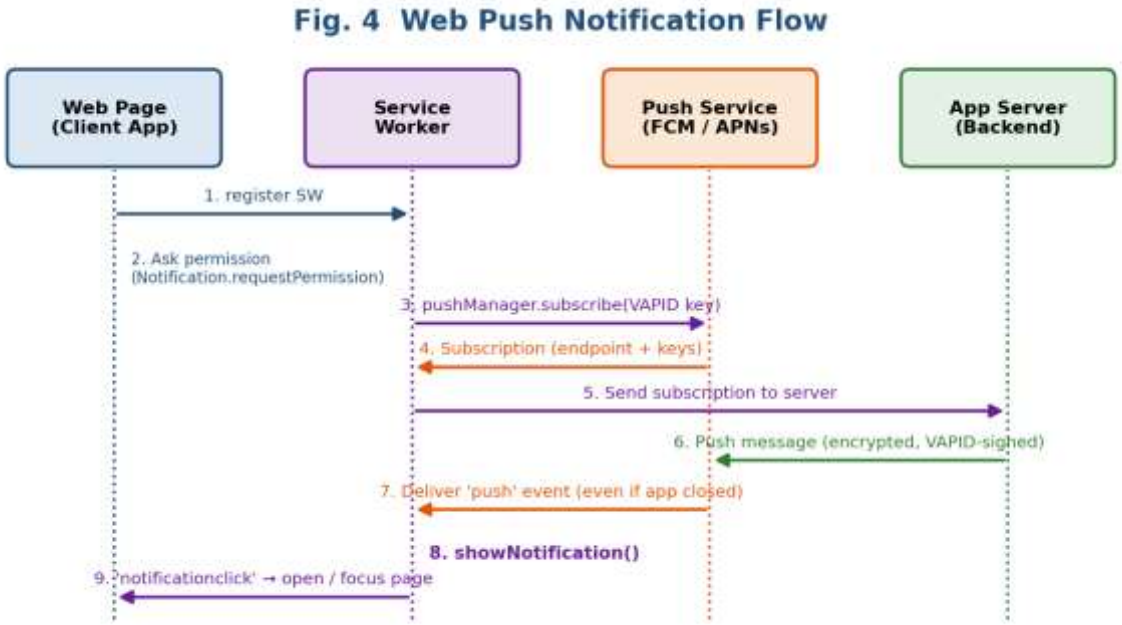


Fig. 6 Web push notification flow

7.1 Steps

1. The page registers the service worker and asks the user for permission using Notification.requestPermission().
2. On approval, the app calls pushManager.subscribe() with the server's public VAPID key; the push service returns a subscription (endpoint URL and encryption keys).
3. The subscription is sent to and stored by the application server.
4. When there is news, the server sends an encrypted, VAPID-signed message to the endpoint of the push service.
5. The push service delivers it to the device, waking the service worker with a push event.
6. The service worker calls showNotification(); when the user taps it, the notificationclick event opens or focuses the app.

```
// Client: subscribe
const reg = await navigator.serviceWorker.ready;
if (await Notification.requestPermission() === 'granted') {
  const sub = await reg.pushManager.subscribe({
    userVisibleOnly: true,
    applicationServerKey: urlBase64ToUint8Array(PUBLIC_VAPID_KEY)
  });
  await fetch('/api/subscribe', { method: 'POST', body: JSON.stringify(sub),
                                  headers: {'Content-Type': 'application/json'} });
}

// sw.js: receive and display
```

```
self.addEventListener('push', e => {
  const data = e.data ? e.data.json() : {title: 'QuickNews', body: 'New update'};
  e.waitUntil(self.registration.showNotification(data.title, {
    body: data.body, icon: '/icons/icon-192.png', data: {url: data.url || '/'} }));
});
self.addEventListener('notificationclick', e => {
  e.notification.close();
  e.waitUntil(clients.openWindow(e.notification.data.url));
});
```

8. Enhanced User Experience – Summary of Roles

Component	Role	User experience benefit
Web App Manifest	Defines name, icons, colours, start URL and display mode; enables installation.	Home-screen icon, splash screen and full-screen native-like look.
Service Worker	Network proxy in the background; manages lifecycle, caching, sync and push.	Offline access, instant loading, resilience on weak networks.
Caching Strategies	Decide cache vs. network for each resource.	Speed for static content and freshness for dynamic content.
Push Notifications	Server-initiated messages displayed by the service worker.	Timely updates and higher re-engagement, even when app is closed.
HTTPS	Secure transport required by service workers.	Trust, privacy and protection from tampering.

Advantages and limitations

- **Advantages:** single code base, no app-store approval, small install size, automatic updates, works offline, lower development cost, discoverable through search.
- **Limitations:** limited access to some device hardware compared with native apps, varying feature support (especially on iOS), and storage quotas managed by the browser.

9. PWA vs Native App vs Traditional Web App

Aspect	Traditional Web App	PWA	Native App
Installation	None (browser tab)	Add to home screen, no store needed	App store download
Offline use	No	Yes (service worker + cache)	Yes
Push notifications	No	Yes (Push API)	Yes
Updates	Automatic	Automatic, in the background	Manual / store approval
Code base	One	One for all platforms	Separate per platform
Device hardware access	Limited	Moderate (camera, GPS, sensors)	Full
Development cost	Low	Low to medium	High
Discoverability	Search and links	Search, links and installable	Store listing

10. Steps to Build a PWA

A PWA can be created by adding a few files to an ordinary responsive website. The overall development workflow is shown below.

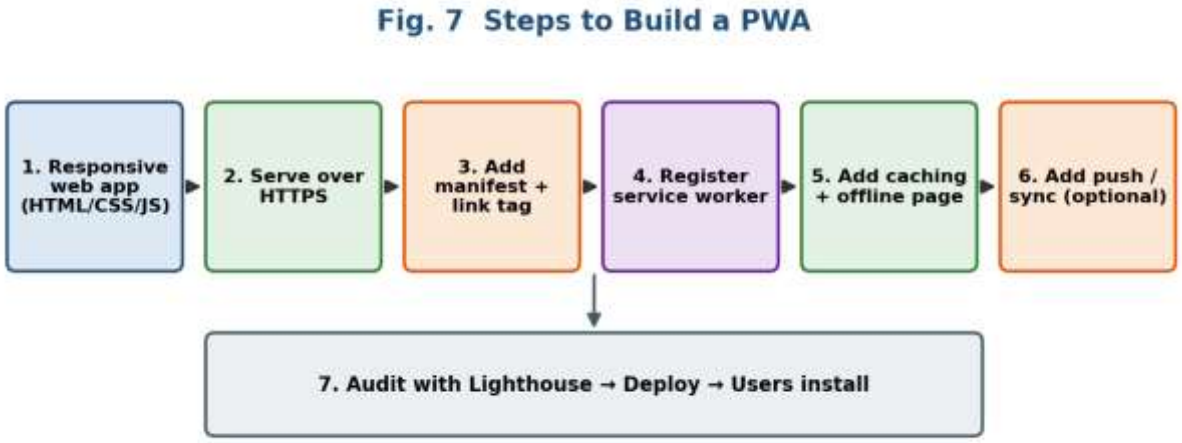


Fig. 7 Steps to build a PWA

Typical project structure:

```
quicknews/
├── index.html          (links manifest, registers service worker)
├── styles.css
├── app.js              (UI logic, SW registration, push subscription)
├── manifest.webmanifest  (name, icons, theme, display)
├── sw.js               (install, activate, fetch, push events)
├── offline.html        (fallback page)
└── icons/              (icon-192.png, icon-512.png)
```

Linking the manifest in index.html:

```
<link rel="manifest" href="/manifest.webmanifest">
<meta name="theme-color" content="#0B5CAD">
<meta name="viewport" content="width=device-width, initial-scale=1">
```

After development, the app is audited with Lighthouse in Chrome DevTools. The installability checks confirm that HTTPS, the manifest, the service worker and offline behaviour are correctly configured.

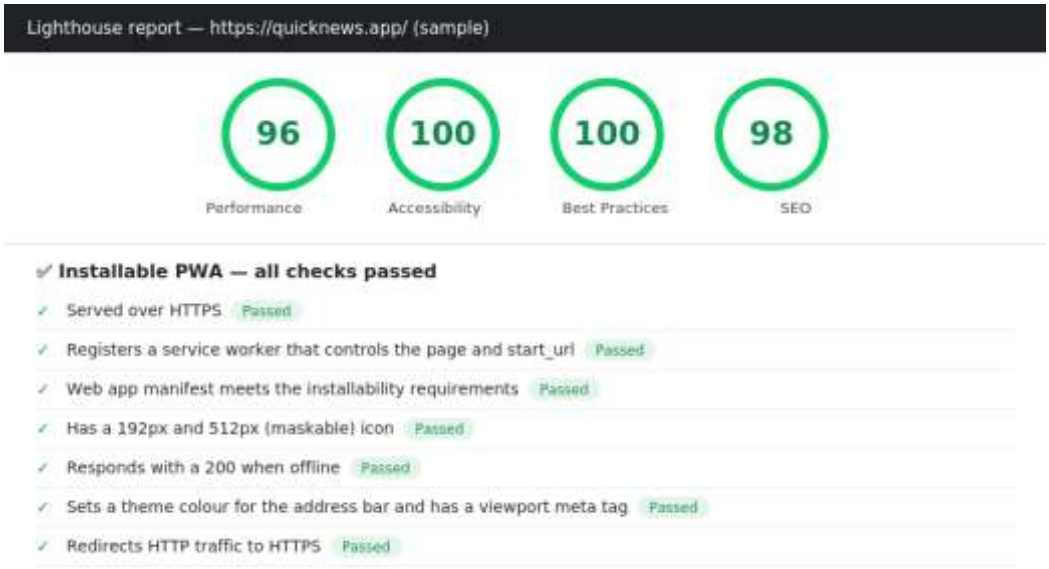


Fig. 8 Sample Lighthouse audit showing PWA installability checks passed

11. Conclusion

Progressive Web Applications combine the reach of the web with the experience of native apps. The Web App Manifest makes the app installable, the service worker supplies offline capability and background processing, caching strategies balance speed and freshness, and push notifications keep users engaged. Together, they provide a fast, reliable, secure and engaging experience across all devices from a single code base.